
Prometheus & Grafana on Kubernetes — Complete Study Notes

1. Prometheus

What It Is

A free, open-source monitoring tool that collects metrics from nodes and stores them in a time-series database.

Why It Exists

To monitor application performance, detect issues, and send alerts automatically.

Think of It Like This

Like a hospital monitor — constantly checking vitals and alarming when something goes wrong.

Real-World Scenario

A DevOps team uses Prometheus to monitor their K8s cluster health and get alerted on Slack when a pod crashes.

Key Points

- Pulls real-time metrics → compresses → stores in time-series database
- Query language: PromQL
- Port: 9090
- Works standalone or with Alertmanager, PagerDuty, Slack, Teams, Email
- Cannot create dashboards — needs Grafana for visualization

Commands

```
# No direct CLI — installed via Helm (see install section)
```

Interview Prep

In practice, Prometheus scrapes (pulls) metrics — it does not receive pushed data. What most people miss is that Prometheus stores data but cannot visualize it — you always pair it with Grafana.

2. Grafana

What It Is

A visualization tool that creates dashboards from data sources like Prometheus.

Why It Exists

Prometheus has raw data but no dashboards — Grafana makes the data readable and visual.

Think of It Like This

Prometheus is the data recorder; Grafana is the screen that displays everything beautifully.

Real-World Scenario

A team connects Grafana to Prometheus and imports a K8s dashboard to monitor CPU and memory in real time.

Key Points

- Datasource = where data comes from (e.g., Prometheus)
- Dashboards = create manually or import
- Port: 3000
- Default credentials: `username: admin password: admin`
- Integrates with PagerDuty, Slack, Email for notifications

Interview Prep

The reason this matters is — Grafana without a connected datasource shows nothing. In practice, always add Prometheus as a datasource first, then import or build dashboards.

3. Helm

What It Is

A package manager for Kubernetes — like `yum` for RedHat or `apt` for Ubuntu.

Why It Exists

Managing raw K8s YAML manifests manually is complex — Helm bundles them into reusable charts.

Think of It Like This

Helm is like an app store for Kubernetes — one command and everything installs.

Real-World Scenario

Instead of writing 10 YAML files for Prometheus, one `helm install` command does it all.

Key Points

Term	Meaning
Chart	Collection of files organized in a directory structure
Chart	Collection of manifest files
Release	A running instance of a chart with a specific config

- Usually in K8S we use manifest file to deploy — with Helm we convert manifest to helm chart and deploy

Interview Prep

What most people miss is the difference between a chart and a release — the same chart installed twice with different configs creates two separate releases.

4. Metrics Server

What It Is

A K8s add-on that collects live CPU and memory usage from nodes and pods.

Why It Exists

Without it, Prometheus cannot scrape resource metrics and `kubectl top` won't work.

Think of It Like This

Metrics Server is the sensor — Prometheus is the dashboard that reads it.

Real-World Scenario

Install Metrics Server before Prometheus so cluster resource metrics are available to scrape.

Key Points

- Installed via single `kubectl apply` from GitHub URL
- Required for HPA (Horizontal Pod Autoscaler) to function
- Must be installed before Prometheus setup

5. KOPS Cluster Setup

What It Is

A tool to create and manage production Kubernetes clusters on AWS.

Why It Exists

You need a running K8s cluster before deploying Prometheus and Grafana.

Think of It Like This

KOPS builds the stadium — Prometheus and Grafana are the screens inside it.

Real-World Scenario

Launch one Amazon Linux 2 EC2 instance, run KOPS commands, and get a full K8s cluster running on AWS.

Key Points

- Launch Amazon Linux 2 instance — name it Prometheus&Grafana Server
- From this server — create K8S Cluster and monitor all those
- Cluster state stored in S3 bucket
- Validate with `kops validate cluster --wait 10m`

Complete Workflow Table

Step	Action	Tool
1	Launch Amazon Linux 2 EC2	AWS
2	Install kubectl + kops	Shell Script
3	Create & validate K8s cluster	KOPS
4	Install Helm	Shell Script
5	Install Metrics Server	kubectl
6	Add Helm repos (Prometheus + Grafana)	Helm
7	Update Helm repos	Helm
8	Create Prometheus namespace	kubectl

9	Install Prometheus via Helm	Helm
10	Create Grafana namespace	kubectl
11	Install Grafana via Helm	Helm
12	Access Grafana UI + connect Prometheus	Browser

FACULTY NOTES — 100% VERBATIM

SYNOPSIS (Verbatim)

PROMETEUS:

its a free & opensource monitoring tool it collects metrics of nodes

it store metrics on time series database we use PromQL language

we can integrate prometheus with tools like pagerduty, slack and email

to send notifications PORT: 9090

GRAFANA:

its a visualization tool used to create dashboard.

Datasource is main component (from where you are getting data) Prometheus will show data but cant create dashboards

Dashboards: create, Import

we can integrate Grafana with tools like pagerduty, slack and email

to send notifications

PORT: 3000

username: admin, password: admin

Prometheus Full Theory (Verbatim)

It can monitor the performance of your applications and services.

it will sends an alert you if there are any issues.

It has a powerful query language that allows you to analyze the data.

It pulls the real-time metrics, compresses and stores in a time-series

database. Prometheus is a standalone system, but it can also be used in conjunction with other tools like Alertmanager to send alerts based on the data it collects. it can be integration with tools like PagerDuty, Teams, Slack, Emails to send alerts to the appropriate on-call personnel. it collects, and it also has a rich set of integrations with other tools and systems.

For example, you can use Prometheus to monitor the health of your Kubernetes cluster, and use its integration with Grafana to visualize the data it collects.

Grafana Full Theory (Verbatim)

we can integrate Grafana with tools like pagerduty, slack and email to send notifications
PORT: 3000
username: admin, password: admin

HELM Full Theory (Verbatim)

HELM:

In K8S Helm is a package manager to install packages

In RedHat: yum

In Ubuntu: apt

In K8S: helm

It is used to install packages on cluster

we can install and deploy applications by using helm

it manages k8s resources packages through charts

chart is a collection of files organized in a directory structure

chat is a collection of manifest files

a Running instance of a chart with a specific config is called Release

Usually in K8S we use manifest file to deploy, but here we will convert manifest to helm chart and deploy

Launch Server + Setup Info (Verbatim)

Launch Amazonlinux2 instance call as Prometheus&Grafana Server,
from this server we will create K8S CLuster and monitor all those

Setup KOPS cluster first, amazon Linux 2

PATH Setup (Verbatim)

```
vim .bashrc
export PATH=$PATH:/usr/local/bin/
source .bashrc
```

kops.sh Script (Verbatim)

```
vi kops.sh

#!/bin/bash
aws configure
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
wget https://github.com/kubernetes/kops/releases/download/v1.25.0/kops-linux-amd64
chmod +x kops-linux-amd64 kubectl
mv kubectl /usr/local/bin/kubectl
mv kops-linux-amd64 /usr/local/bin/kops

#vim .bashrc
#export PATH=$PATH:/usr/local/bin/
#source .bashrc
```

KOPS Cluster Create (Verbatim)

```
configuration Status=Enabled
export KOPS_STATE_STORE=s3://reyaz-kops-testbkt123.k8s.local
```

```
kops create cluster --name reyaz.k8s.local --zones ap-south-1a
--master-count=1 --master-size t2.medium --node-count=2
--node-size t2.micro
kops update cluster --name reyaz.k8s.local --yes --admin
```

Run the below commands (Verbatim)

```
export KOPS_STATE_STORE=s3://reyaz-kops-testbkt123.k8s.local
kops validate cluster --wait 10m
kops update cluster --name reyaz.k8s.local --yes --admin
kops rolling-update cluster
```

Delete Cluster (Verbatim)

```
# if you want to delete :
kops delete cluster --name reyaz.k8s.local --yes
```

Install HELM (Verbatim)

```
curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
helm version
```

Install Metric Server (Verbatim)

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/high-availability-1.21+.yaml
```

Add Helm Repositories (Verbatim)

```
# First add helm repositories
helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
```

Update Helm Chart Repos (Verbatim)

```
helm repo update  
helm repo list
```

Create Prometheus Namespace (Verbatim)

```
kubectl get ns  
kubectl create namespace prometheus  
kubectl get ns
```

Install Prometheus (Verbatim)

```
helm install prometheus prometheus-community/prometheus  
--namespace prometheus --set  
alertmanager.persistentVolume.storageClass="gp2" --set  
server.persistentVolume.storageClass="gp2"
```

```
kubectl get pods -n prometheus  
kubectl get all -n prometheus
```

Create Namespace Grafana (Verbatim)

```
kubectl create namespace grafana
```

Install Grafana (Verbatim)

```
helm install grafana grafana/grafana --namespace grafana --set  
persistence.storageClassName="gp2" --set  
persistence.enabled=true --set adminPassword='Root123456'  
--set service.type=LoadBalancer
```

```
kubectl get pods -n grafana  
kubectl get service -n grafana
```

Access Grafana (Verbatim)

```
Copy the Instance-IP and paste in browser  
username: admin, password = Root123456
```

```
Check SG, nodes and ELB
```

